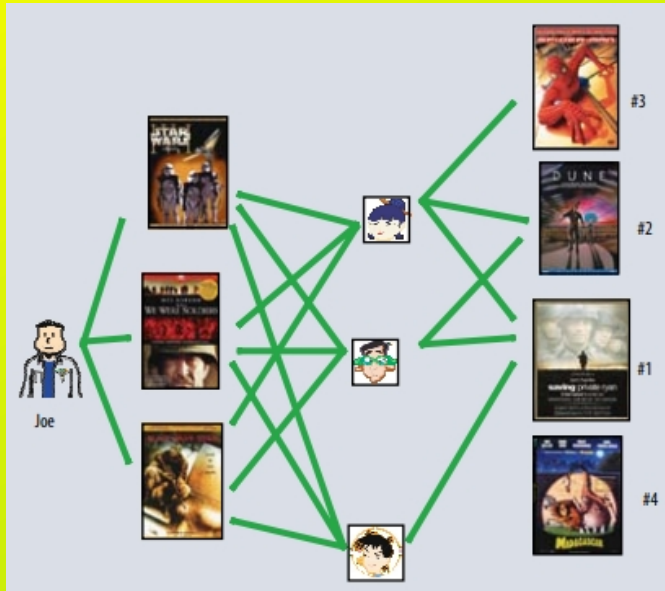


Sistemas de Recomendación

Agenda



01 Definición del problema y abordaje

02 Arquitectura

03 Memory-Based Collaborative Filtering

04 Caso de estudio: Netflix

Definición

Los sistemas de recomendación (SR) son técnicas para sugerir ítems apropiados a un usuario*.

Se aplican en contextos donde un usuario por sí mismo no puede evaluar correctamente qué consumir (qué se puede consumir es un “ítem”).

En este contexto, existen sugerencias no personalizadas y sugerencias personalizadas.

Las no personalizadas suelen ser más sencillas de hacer y podemos encontrar en este grupo a, por ejemplo, las siguientes reglas de negocio:

“Las comedias más buscadas”

“Los ítems más populares este mes”

“Programas estadounidenses basados en libros” (tomado de Netflix)

En el caso de las sugerencias personalizadas, nuestro interés es rankear los ítems intentando que a mayor ranking mayor sea la preferencia de un usuario por el mismo.

Para ello, los SR capturan información sobre las preferencias de los usuarios, esto puede ser explícito o implícito:

Explícito: este es el caso en que un usuario puntúa un producto, sea con estrellas, con números, con una “manito” arriba o abajo, o un “me gusta” o “no me gusta”, etc.

Implícito: en este caso se infieren las preferencias a partir de las acciones, por ejemplo, si un usuario realizó un clic en un determinado producto, vio un capítulo de una serie, vio los detalles de una descripción, etc, entonces infiero que tiene cierto interés en el producto.

Arquitectura

Los sistemas de recomendación involucran varios pasos además del modelo de recomendaciones en sí. Se presenta una posible esquematización:

Recolección de los datos: esto involucra tomar información sobre las preferencias, sobre las características de los usuarios y sobre los items.

Modelo de recomendaciones: aquí se encuentra el modelo que genera la personalización del contenido.

Post procesamiento de la recomendación: en este punto se suelen incluir ciertas reglas de negocio que permiten mejorar las sugerencias evitando sugerencias inútiles.

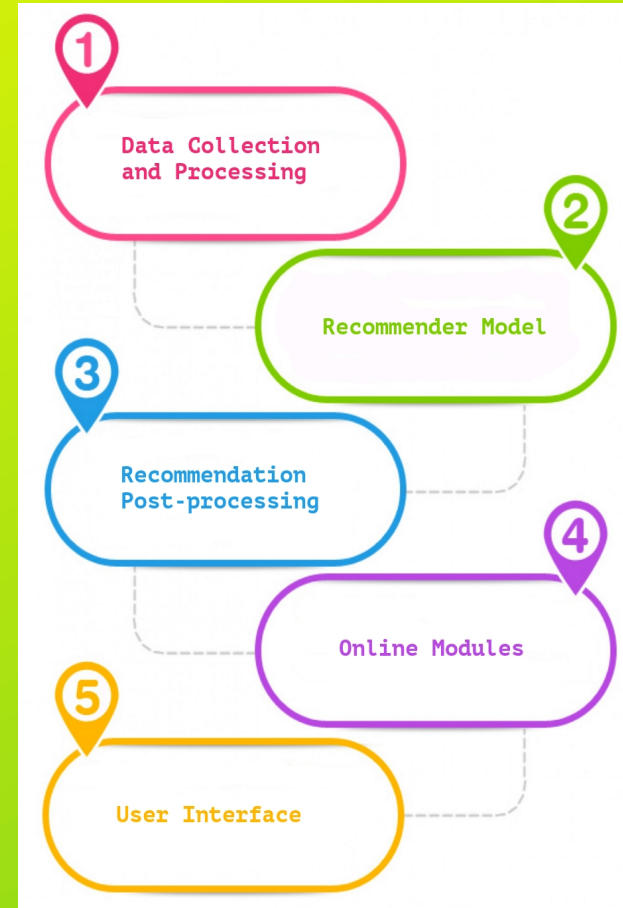
En E-commerce: no recomendar un producto si no se cuenta con el stock o si es de otro país.
En streaming: no sugerir una película que ya fue vista o que ya no es parte del contenido de la página.

En redes sociales: evitar sugerir perfiles que no se pueden ver públicamente o que es la misma persona pero con otro perfil.

Módulos online: una vez que se cuenta con la recomendación hay que poder entregarla eficientemente a los usuarios cuando éstos las requieren.

Generalmente, ésto involucra distintos servicios que en real time deben poder responder las necesidades de las plataformas.

Interfaz de usuario: finalmente las recomendaciones se entregan en una interfaz gráfica, el front-end de una app.



Enfoques

Los sistemas de recomendación tradicionalmente se agrupan en:

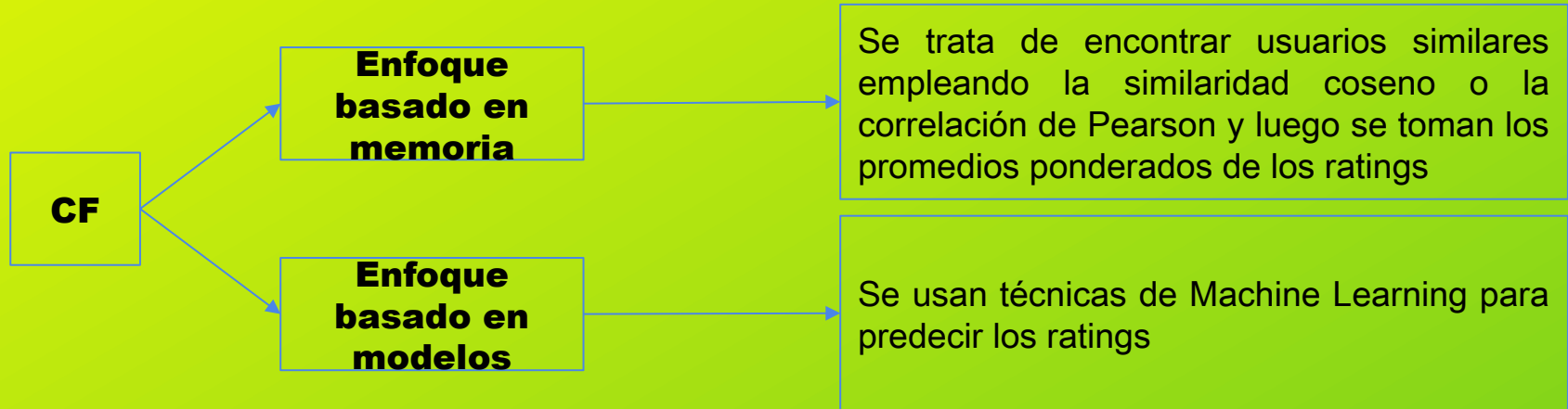
- **Basados en popularidad**: en este caso no hay una gran personalización.
- **Collaborative filtering** (filtro colaborativo)
- **Content-based filtering** (filtro basado en contenido)
- **Híbridos**, combinando diversos enfoques

Además, existen enfoques más recientes que emplean técnicas de deep learning.

Filtro Colaborativo

El filtro colaborativo se basa en las siguientes ideas:

1. Los usuarios dan ratings al catálogo de ítems (de manera explícita o implícita).
2. Los patrones en los datos me pueden ayudar a predecir los ratings porque:
 - Hay clientes con preferencias similares
 - Hay características latentes en los ítems que influyen los ratings de los usuarios.



Filtrado Colaborativo basado en Memoria

Podemos distinguir dos tipos de filtros:

- **User-item filtering**: se toma a un usuario, se encuentran usuarios similares y se recomiendan ítems que a esos usuarios similares les gustaron. En este caso el input es un usuario y el output es un ítem.
Típicamente, se explican como “A usuarios que son similares a vos también les gustó...”.
- **Item-item filtering**: se toma un ítem, se encuentran usuarios que les haya gustado ese ítem, y se buscan otros ítems que a esos usuarios (o usuarios similares) les haya gustado. Aquí el input es un ítem y el output es un ítem.
En general se explican como “A usuarios que les gustó ... también les gustó ...”.

Vamos a entender cómo se calcula cada uno y por qué el item-item filtering (o item-based filtering) es preferido por sobre el user-item filtering (o user-based filtering).

Filtrado Colaborativo basado en Memoria

Nuestro problema es, dada la siguiente matriz, predecir los valores faltantes, **rankear** las mejores sugerencias y **sugerir los K ítems** con mayor ranking para alguno de los usuarios.

Matriz usuario-item

	The Name of the Wind	The Lord of the Rings	Introduction to Statistical Learning	Brave New World	A Study in Scarlet
Nico	5	4	4	?	?
Leo	?	3	5	5	5
Fran	4	5	5	?	3

Similitud Coseno

Antes de ver cómo realizar una recomendación, va a ser necesario entender la **similitud coseno**. A nosotros nos va a importar saber qué tan diferentes son los usuarios o qué tan diferentes son los items y para ello necesitamos una **medida de distancia**.

Para calcular la distancia entre los usuarios (o entre los items) podemos pensarlos como vectores en el espacio. Por ejemplo, podemos querer comparar un usuario que puntuó (A=5, B=4, C=5), con un usuario que puntuó (A=5, B=3, C=2).

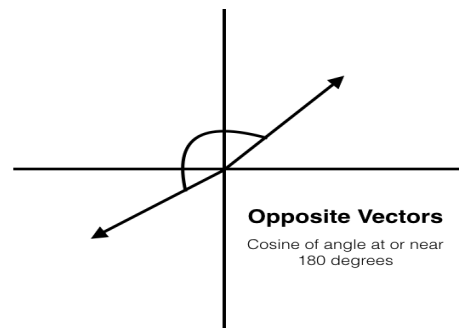
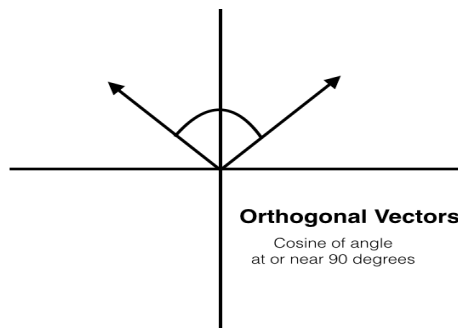
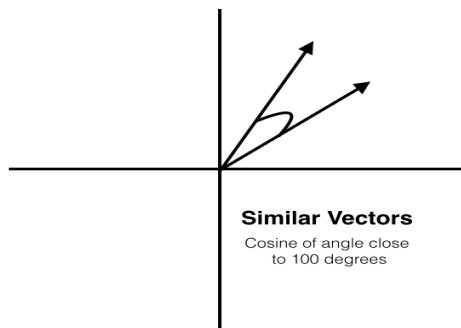
Sin embargo, no nos interesa tomar las cantidades en consideración sino la proporción. Es decir, no nos importa la magnitud de los vectores, sino su dirección y sentido. Por este motivo, vamos a emplear la similitud coseno.

La similitud coseno se calcula de la siguiente manera:

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}},$$

Como se ve, se requiere sumar los componentes de cada uno de los vectores para la posición i , y dividirlo por el producto de los módulos de los vectores.

La similitud coseno capta el ángulo entre ambos vectores.



Filtrado colaborativo por Usuario-Item

Ahora bien, veamos cuáles son los pasos a seguir para poder realizar un filtro User-Item (o user-based)

En este caso los pasos a seguir son:

- 1- Calcular la similaridad entre los usuarios
- 2- Predecir los ratings de los ítems que el usuario X no puntuó
- 3- Seleccionar K mejores ítems

1- Calcular la similaridad entre los usuarios

Como vimos antes, vamos a emplear la similaridad coseno.

2- Predecir los ratings

Una vez que calculamos las similitudes de un usuario contra los demás necesitamos generar la predicción. Para ello lo que hacemos es un promedio de las puntuaciones de los usuarios que puntuaron ese ítem ponderado por la similitud. Luego, multiplicamos por un factor de normalización.

Dados 'n' usuarios U que hicieron ranking R_u sobre un cierto ítem con similitudes entre usuarios S_u , la predicción para R de ese ítem será:

$$R_U = \left(\sum_{u=1}^n R_u * S_u \right) / \left(\sum_{u=1}^n S_u \right)$$

Filtrado colaborativo por Usuario-ítem

1- Calcular la similaridad entre los usuarios

Como vimos antes, vamos a emplear la similaridad coseno. Calculemos cuánto nos da la distancia entre Nico y Fran. Recordemos que : Los ratings de Nico fueron (5, 4, 4, ?, ?) y los de Fran (4, 5, 5, ?, 3). Los casos que en alguno de los dos tiene un nulo no los podemos incluir, entonces y, siguiendo la fórmula vista, la similitud se calcula como:

$$(5*4 + 4*5 + 4*5)/(np.sqrt(5**2+4**2+4**2)*np.sqrt(4**2+5**2+5**2)) = 0.978$$

Así, calculamos las similitudes de Nico contra todos los demás.

2- Predecir los ratings

¿Cómo lo aplicamos en nuestro caso?

Supongamos que queremos predecir A Study in Scarlet (Study) para Nico, lo que tenemos que hacer es entonces:

$$R(\text{Nico}, \text{Study}) = K \times [\text{sim}(\text{Nico}, \text{Leo}) \times R(\text{Leo}, \text{Study}) + \text{sim}(\text{Nico}, \text{Fran}) \times R(\text{Fran}, \text{Study})]$$

$$\text{Con } K = 1 / [|\text{sim}(\text{Nico}, \text{Leo})| + |\text{sim}(\text{Nico}, \text{Fran})|]$$

Si realizamos las cuentas nos da:

$$R(\text{Nico}, \text{Study}) = K \times [0.97 \times 5 + 0.98 \times 3] \text{ con } K = 1 / [0.97 + 0.98]$$

$$R(\text{Nico}, \text{Study}) = 0.51 \times 7.79 = 4$$

3- Elegir los k mejores libros

En este caso tenemos pocos libros, pero si tuviéramos muchos ítems deberíamos elegir los k mejores y mostrar solamente eso.

Filtrado colaborativo por Item-item

Los pasos a seguir para poder realizar un filtro Ítem-item (o ítem-based) son:

- 1- Calcular la similaridad entre los ítems
- 2- Predecir los ratings de los ítems que el usuario X no puntuó
- 3- Seleccionar K mejores ítems

1- Calcular la similaridad entre los ítems

Como vimos antes, vamos a emplear la similaridad coseno.

2- Predecir los ratings

Una vez que calculamos las similitudes de un ítems contra los demás necesitamos generar la predicción. Para ello lo que hacemos es un promedio de las puntuaciones de los usuarios que puntuaron ese ítem ponderado por la similitud entre ítems. Luego, multiplicamos por un factor de normalización.

Dados ‘n’ usuarios que tienen sus ratings sobre ítems y ‘m’ ítems, se calculan las similaridades entre ítems (con la matriz de similitudes) y la predicción para el ítem no rankeado será:

$$R_U = \left(\sum_{u=1}^n R_u * S_j \right) / \left(\sum_{j=1}^m S_j \right)$$

Filtrado colaborativo por ítem-ítem

1.1- Transponer la matriz

	Nico	Leo	Fran
The Name of the Wind	5	?	4
The Lord of the Rings	4	3	5
Introduction to Statistical Learning	4	5	5
Brave New World	?	5	?
A Study in Scarlet	?	5	3

Filtrado colaborativo por ítem-ítem

1.2- Calcular matriz de similitudes haciendo el producto de $M \times M$ (traspuesta)

	The Name of the Wind	The Lord of the Rings	Introduction to Statistical Learning	Brave New World	A Study in Scarlet
The Name of the Wind					
The Lord of the Rings	0.975				
Introduction to Statistical Learning	0.975	0.975			
Brave New World	?	1	1		
A Study in Scarlet	1	0,88	0,97	1	

Filtrado colaborativo por ítem-ítem

2- Predecir los ratings

Una vez que calculamos las similitudes entre ítems podemos aplicar la misma fórmula que antes pero pensando el problema al revés.

Volvamos al caso de predecir A Study in Scarlet para Nico. Ahora en vez de hacerlo partiendo de los ratings de otras personas para Study lo vamos a hacer tomando los ratings de los ítems que Nico sí puntuó y viendo las similitudes entre estos ítems y Study. Entonces sería:

$$R(\text{Nico}, \text{Study}) = K \times [\text{sim}(\text{Name}, \text{Study}) \times R(\text{Nico}, \text{Name}) + \text{sim}(\text{Lord}, \text{Study}) \times R(\text{Nico}, \text{Lord}) + \text{sim}(\text{Statistical}, \text{Study}) \times R(\text{Nico}, \text{Statistical})]$$
 con $K = 1 / [|\text{sim}(\text{Name}, \text{Study})| + |\text{sim}(\text{Lord}, \text{Study})| + |\text{sim}(\text{Statistical}, \text{Study})|]$

$$R(\text{Nico}, \text{Study}) = K \times [1 \times 5 + 0,88 \times 4 + 0,97 \times 4]$$
 con $K = 1 / [1 + 0,88 + 0,97]$

$$R(\text{Nico}, \text{Study}) = 4,35$$

3- Elegir los k mejores ítems

Como en el user-based filtering tendremos que elegir los k mejores ítems.

Item-based vs User-based collaborative filtering

¿Qué ventajas tiene uno sobre otro?

La principal ventaja de IBCF contra UBCF es que en el primer caso se puede computar la matriz de similitudes de items de manera offline y tener precalculado ese paso. Esto permite luego generar predicciones online de manera mucho más rápida que en el caso de UBCF.

No es conveniente realizar este proceso de cómputo de similitudes offline en el caso de los usuarios porque:

- 1- La cantidad de usuarios suele ser mucho mayor que la de items. Computar ésto haría al proceso muy costo.
- 2- Cuando un usuario puntúa un item, sus similitudes con otros usuarios se ven drásticamente afectadas, ya que un usuario suele puntuar pocos items. Por otra parte, esto afecta poco a la matriz de items a items, ya que cada item es probable que haya recibido puntuaciones de diversos usuarios.

Modelos basados en contenido

- Se basan en información del contenido en lugar de opiniones o interacciones.
- Usan un algoritmo o método de extracción de features para inducir preferencias del usuario
- El enfoque apunta a recomendar ítems similares a los que le gustaron anteriormente

¿Qué es el contenido?

Pueden ser atributos o características del ítem: género, actores, año. O contenido textual, título, descripción, tablas de contenido con las que usar métricas de similitud entre texto o de procesamiento de lenguaje (NLP).

Los features se pueden extraer de la señal misma (audio, imagen)

Más común para productos de texto (ej. noticias), que pueden usar features como palabras clave (*keywords*) para recomendar mediante la comparación del modelo del usuario (palabras preferidas) y el contenido (keywords que aparecen en el texto)

Modelos basados en contenido

Ventajas:

- No necesita data de otros usuarios
- Puede recomendar a perfiles poco frecuentes
- Puede recomendar ítems nuevos
- Puede proveer una explicación de la recomendación listando los features que fueron relevantes

Desventajas:

- Requiere construir features significativos, y posiblemente técnicas más sofisticadas para su extracción
- Los gustos del usuario tienen que poder ser representadas como una función de esos features
- No aprovecha juicios calificados de otros usuarios
- Fácil de sobreajustar (por ej con un usuario con pocos datos, podemos estancarlo en un solo tema, o potenciar un *efecto burbuja*).

Ejemplo extracción de features: TF-IDF

Una manera sencilla de reflejar los temas de los que trata un documento es mediante palabras clave o el vocabulario contenido.

La representación más sencilla es una matriz con el número de cada término en cada documento.

Matriz de términos por documento

	D1	D2	D3
el	1	1	1
árbol	1	0	1
crece	0	0	1
tenedor	0	1	0
variable	0	0	0

D1: el árbol

D2: el tenedor

D3: crece el árbol

Ejemplo extracción de features: TF-IDF

- La representación clásica en Information Retrieval de TF-IDF (Term-Frequency Inverse Document Frequency) construye sobre esta usando la frecuencia de cada término, y dividiéndola por el número de documentos en las que aparece.
- tf es la frecuencia de un término sobre el total de palabras en el mismo
- La idf nos indica qué tan informativo es un término en un documento, ya que para una palabra que aparece en todos el valor será bajo.

$$tfidf(t, d, D) = tf(t, d) \times idf(t, D)$$

$$idf(t, D) = \log \frac{|D|}{1 + |\{d \in D : t \in d\}|}$$

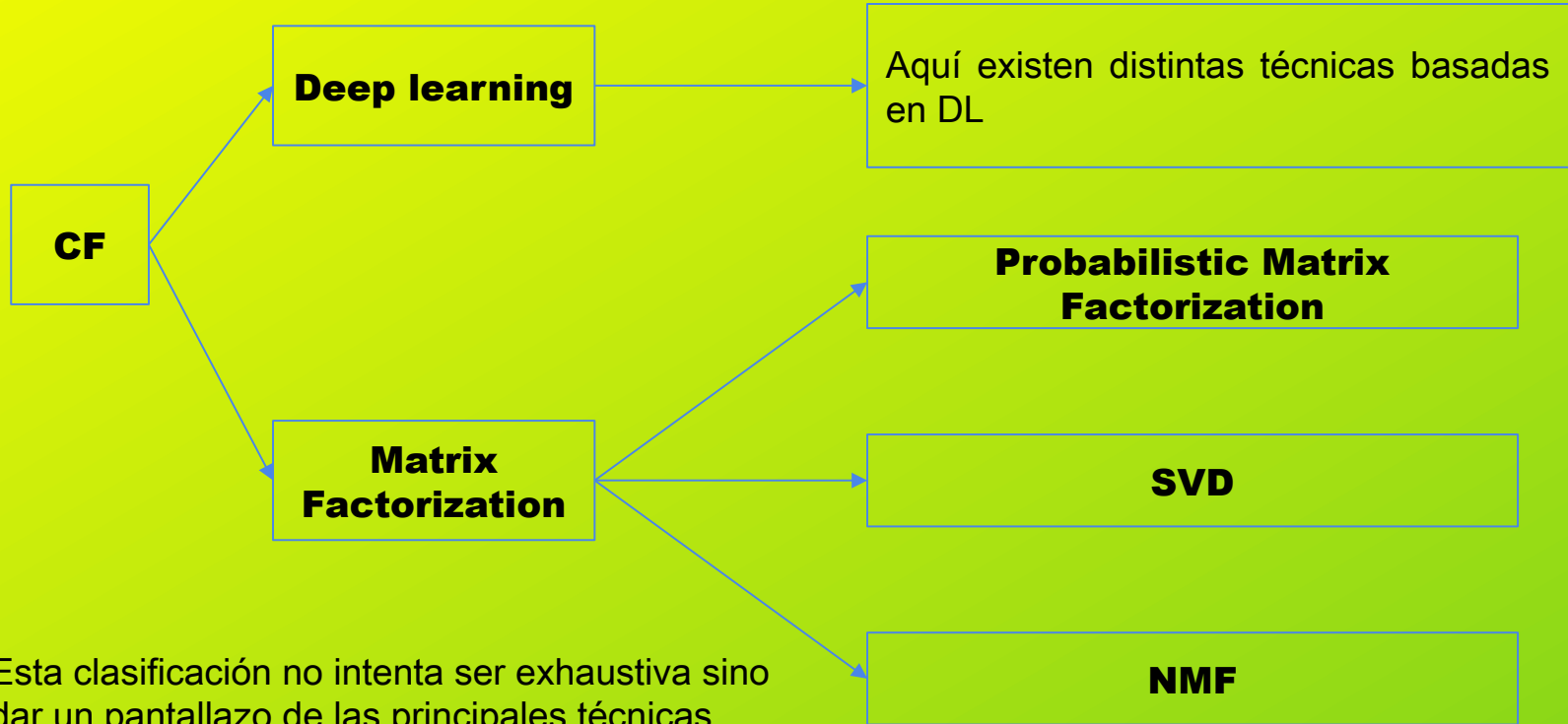
- El numerador D es el espacio de documentos ($d_1, d_2, \dots, d_n$) siendo n el número de documentos.
- El denominador implica la cantidad total de documentos en los cuales apareció el término t .

Llamando $\text{contenido}(i)$ a los atributos TFIDF de un ítem, y $\text{perfil}(u)$ los pesos para cada atributo TFIDF según los ítems preferidos de ese usuario, podemos definir una función de utilidad como

$$u(i, u) = \text{puntaje}(\text{contenido}(i), \text{perfil}(u))$$

Nuestra métrica en este caso puede ser la similitud coseno, resultando en: $\text{sim}(\text{TFIDF}(i), \text{TFIDF}(u))$

Filtrado colaborativo basado en modelos



Filtrado colaborativo por SVD

SVD

SVD (Singular Value Descomposition) es un método de factorización algebraica que tiene diversas aplicaciones, una de las aplicaciones más famosas es para SR.

Dado una matriz A se puede demostrar que:

$$A = U \Sigma V^T$$

Donde U y V son ortogonales y Sigma es una matriz diagonal de valores singulares. Los valores singulares de A equivalen a la raíz de los autovalores de A:

$$\sigma_i = \sqrt{\lambda_i}$$

Σ por ser una matriz diagonal opera como escalando una de las matrices vecinas, por este motivo vamos a suponer que la juntamos con alguna de ellas.

Entonces, ahora podemos reescribir nuestro problema como:

$$R = P \cdot Q$$

Cada fila de P es específica al usuario 'u' y cada columna de Q es específica al item 'i'

Filtrado colaborativo por SVD

Resolver el problema anterior es imposible si la matriz es incompleta, lo cual es el caso normal ya que los usuarios nunca puntúan todos los items. Por este motivo se replantea el problema de la siguiente manera:

$$\hat{r}_{ui} = \mu + b_i + b_u + p_u q_i$$

Donde ' $\mathbf{r}$ ' es la predicción del ranking, para el usuario $\mathbf{u}$ y el item $\mathbf{i}$, ' μ ' es un parámetro constante de sesgo, b_i es un sesgo para el item $\mathbf{i}$, b_u es un sesgo para el usuario $\mathbf{u}$. $p_u q_i$ es la factorización de la matriz original.

Para encontrar los parámetros se emplea descenso por el gradiente y se minimiza la suma de errores cuadráticos. Además, en el problema se agrega un parámetro de regularización. Sólo se toman en consideración los errores de casos completos, los casos nulos son omitidos para el ajuste de los parámetros. Entonces el problema a resolver es:

$$\text{Min} \sum (r_{ui} - \mu - b_i - b_u - q_i^T p_u)^2 + \lambda (b_i^2 + b_u^2 + \|q_i\|^2 + \|p_u\|^2)$$

$\hat{r}_{ui}$

hiperparámetro de regularización

Implicit Feedback

Una dificultad general es que la mayoría del tiempo, las personas no proveen puntajes en absoluto, por lo que la data es escasa. Tomemos por ejemplo YouTube, probablemente pases por muchos más vídeos de los que puntuas.

En contraste al filtro colaborativo por feedback explícito, donde el usuario nos da puntajes directos como ratings, likes, o reseñas, en este caso se estima la preferencia usando el **comportamiento**

Algunos ejemplos son historial de compras, de navegación, patrones de búsqueda, movimientos del mouse o el tiempo que pasó observando un contenido.

Implicit Feedback

En el caso anterior para feedback **explícito** teníamos como función de pérdida (sin los biases):

$$L_{exp} = \sum_{u,i \in S} (r_{ui} - \mathbf{q}_u^\top \cdot \mathbf{p}_i)^2 + \lambda_q \sum_u \|\mathbf{q}_u\|^2 + \lambda_p \sum_u \|\mathbf{p}_i\|^2$$

$\hat{r}_{ui}$ Regularización

Para el feedback implícito (como fue propuesto en uno de sus trabajos fundacionales) tenemos:

$$L_{imp} = \sum_{u,i} c_{ui} (t_{ui} - \mathbf{p}_u^\top \cdot \mathbf{q}_i)^2 + \lambda_p \sum_u \|\mathbf{p}_u\|^2 + \lambda_q \sum_u \|\mathbf{q}_i\|^2$$

Confianza

$t \in 0, 1$

Matriz de preferencias

Implicit Feedback

Reemplazamos los ratings por una matriz de preferencias, t . T va a ser 1 si el usuario interactuó con ese ítem, y 0 si no.

A su vez, agregamos la matriz c que va a ser la matriz de confianza (*confidence*) reflejando la incertidumbre de que el usuario u tiene la preferencia

$$c_{ui} = 1 + \alpha d_{ui}$$

Donde α es un hiperparámetro a tunear por cross-validation y ‘ d ’ es un indicador de la interacción como cantidad de clicks, o un valor binario en caso de “likes”

Complicaciones: Cold Start

¿Qué significa?

Cold start es el problema de no saber qué recomendarle a un usuario la primera vez o de contar con un ítem nuevo que aún no ha sido puntuado.

Para resolver esto existen distintas estrategias:

- Recomendaciones basadas en el contenido. En este caso se intenta buscar productos con las mismas características, calculando la similitud entre los productos. Cómo medir esta similitud va a variar de acuerdo al problema, en los documentos se puede hacer calculando la similitud coseno entre los vectores de palabras.
- Pedirle al usuario que nos dé cierta información sobre gustos.
- Dar una sugerencia basada en la popularidad de los ítems en la plataforma.
- Usar la información de contexto: ¿desde dónde se conecta, cuál es su género, etc?

Complicaciones: Dispersión (sparsity)

Típicamente, se tienen grandes cantidades de productos, y un usuario puntúa un pequeño porcentaje

Por ejemplo en Amazon, hay millones de libros, y un usuario puede haber comprado algunas decenas. La probabilidad de que dos usuarios que compraron 100 libros tengan uno en común, entre un catálogo de 1 millón, es de 0.01.

Por ejemplo en los datos compartidos por Netflix, en la matriz de usuarios/peliculas se tiene:

- $500.000 \times 17.000 = 8500$ millones de valores, de los cuales solo 100M no son 0s

Para afrontar esto podemos usar:

- Factorización de matrices
- Clustering
- Proyección (PCA, SVD...)

Complicaciones: Escalabilidad

Un algoritmo como vecinos más cercanos (KNN) crece en costo computacional con el número de usuarios y de productos.

Con millones de usuarios y productos un servicio web puede tener problemas para escalar

En este caso la complejidad worst-case es $O(m*n)$, o $O(m+n)$ si en la práctica se consideran pocos productos por usuario. Para ayudar podemos clusterizar reduciendo el espacio de salida.

- En CF basado en usuarios, la similitud es dinámica, no podemos precomputar los más cercanos.
- En cambio en ítem-ítem, podemos precomputar las similitudes entre los mismos ahorrando tiempo al momento de la recomendación.

Evaluación

En SR se emplean métricas típicas como **accuracy**, **mae**, **rmse**, etc. Por otra parte, existen métricas específicas entre las que vale la pena mencionar a **FCP** (fraction of concordant pairs o fracción de pares concordantes).

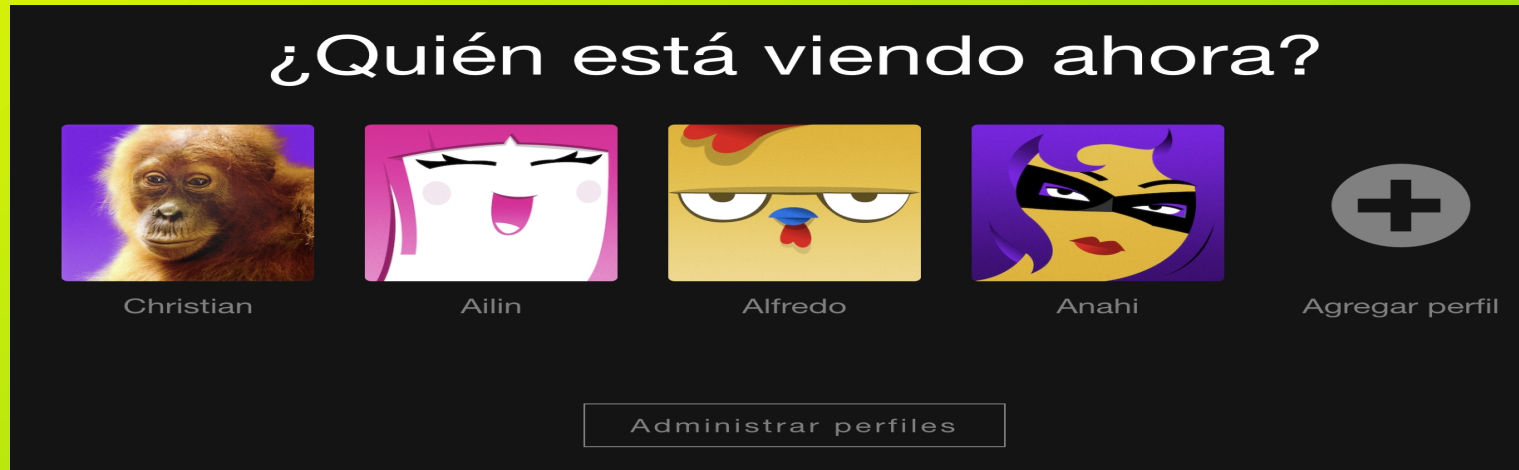
FCP es la proporción de pares de items que fueron correctamente rankeados, es decir, que son concordantes, sobre el total de pares. Esto es, cuando para un usuario u , un item i y un item j se cumple que: $\hat{r}_{ui} > \hat{r}_{uj}$ y que $r_{ui} > r_{uj}$

Caso: Netflix

Ingreso

Es importante entender que no sólo el modelo del sistema de recomendación es relevante para el usuario sino todo el contexto, es decir, la UX completa. Por este motivo nos detendremos brevemente a entender la plataforma de Netflix.

En primer lugar, la plataforma nos requiere identificar qué perfil vamos a usar. Esto va a permitirle a la plataforma cargar los datos históricos del usuario.

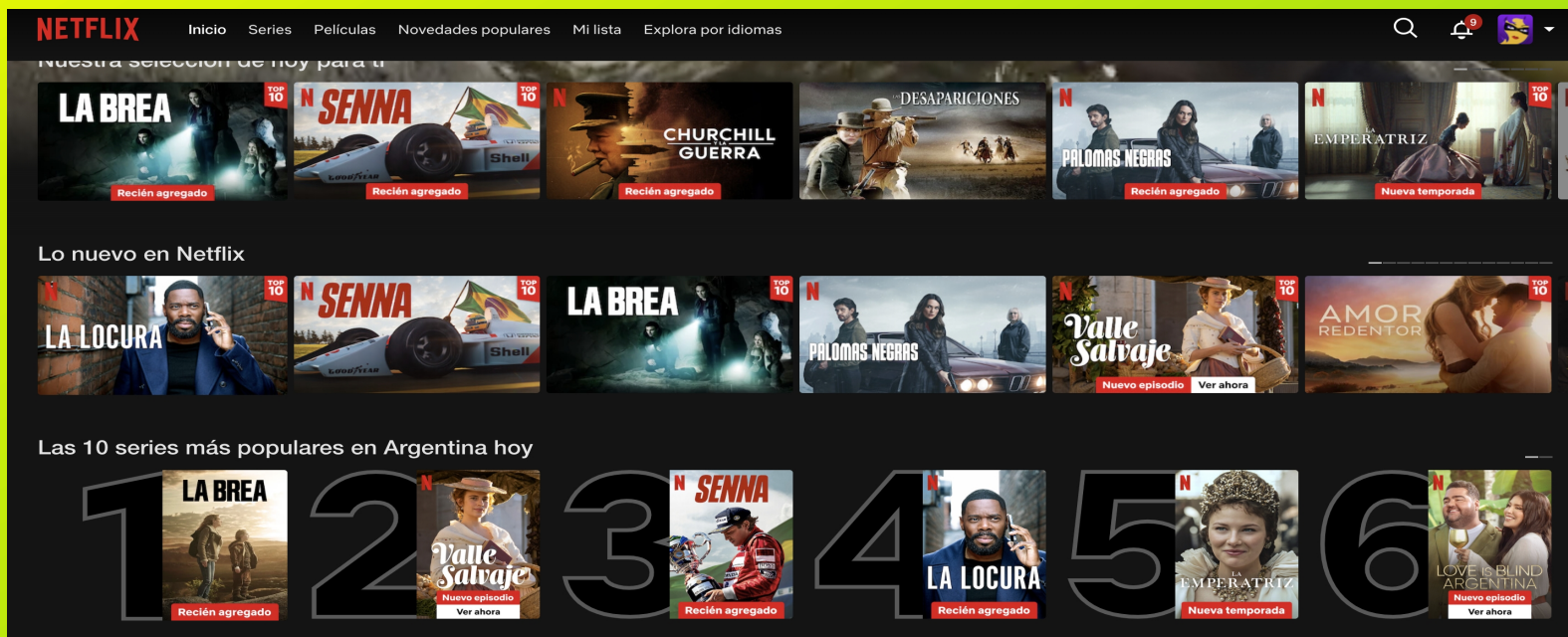


Caso: Netflix

Pantalla principal

En esta pantalla es donde Netflix nos sugiere qué producciones podrían ser de nuestro interés.

Aquí es donde se mezclan las recomendaciones personalizadas que surgen de los modelos y las que provienen de reglas de negocios, así como otras recomendaciones no personalizadas.



Caso: Netflix

Sugerencias sin modelos

Sugerencia de seguir viendo algo que el usuario ya vio

Continuar viendo contenido de Anahi



Lista que el usuario mismo genera

Mi lista



Caso: Netflix

Basados en popularidad:

Si bien parecen genéricos, están personalizados

Popular



Tendencias

Series aclamadas por la crítica



Caso: Netflix

Según recencia

También están personalizados

Recientemente agregados

Estrenos de esta semana



Nuevos lanzamientos

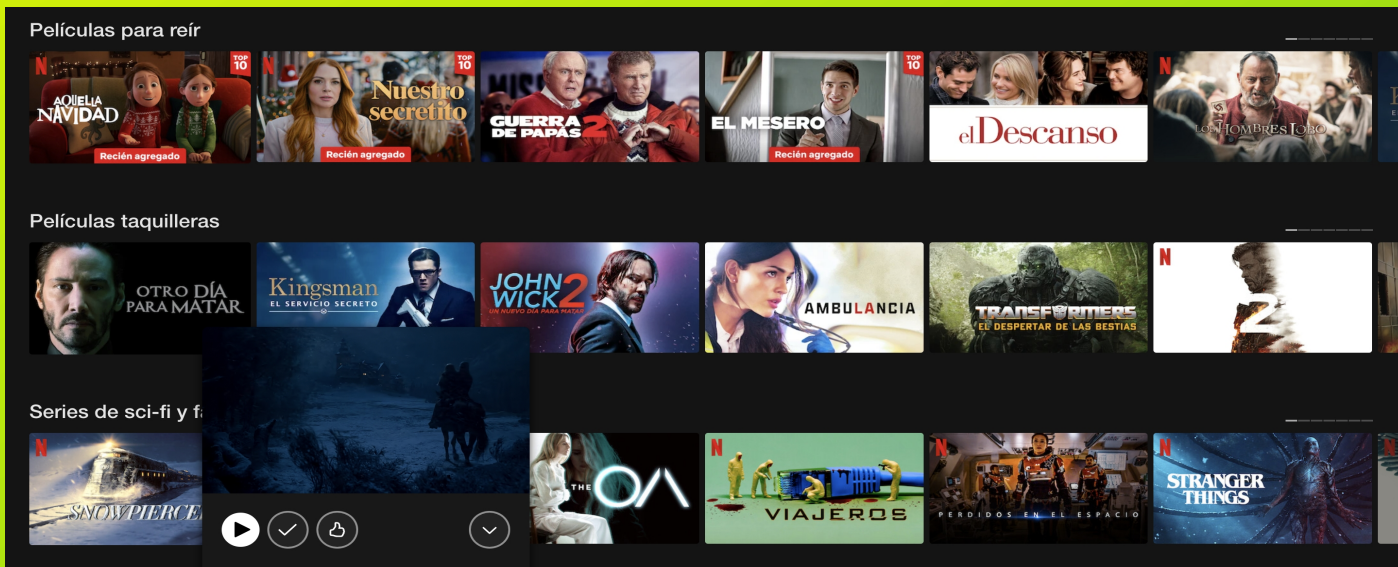
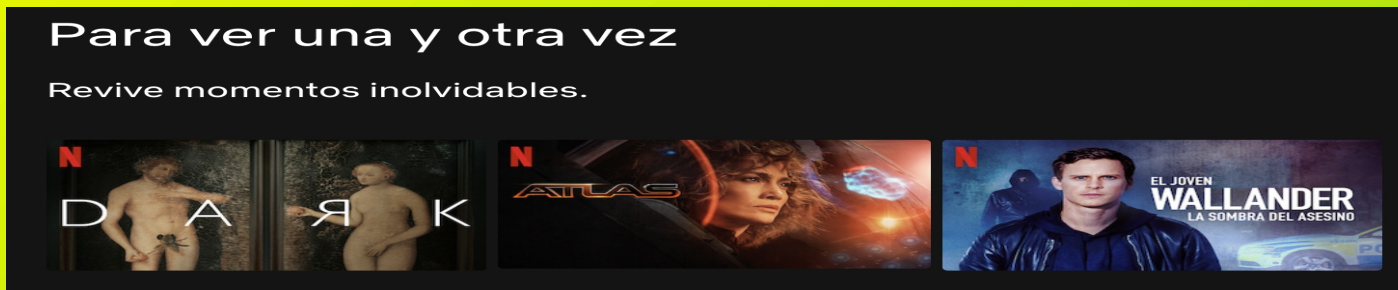
Lo nuevo en Netflix



Caso: Netflix

Porque viste...

Netflix permite buscar en géneros y subgéneros y allí ordena las producciones basándose en algún motor de recomendación



Conclusión

Existen técnicas de variada complejidad para sistemas de recomendación, desde heurísticos basados en álgebra sencilla hasta extracción de features con deep learning.

Los sistemas de recomendación tienen aplicación en multiplicidad de áreas, y dada la enorme cantidad de información disponible toman cada vez más relevancia en la sociedad general.

Usando factorización de matrices, machine learning y otras herramientas facilitadas mediante Python podemos construir fácilmente sistemas para sugerir ítems basados en texto, características estructuradas, o también imagen, video y audio ya sea mediante comportamiento de otros usuarios por CF o extracción de features para uso de modelos o enfoques híbridos.

Referencias

MATRIX FACTORIZATION TECHNIQUES FOR RECOMMENDER SYSTEMS

Andrew Ng Videos

Awesome RecSys (compilado)

Recommender System Handbook (2015, Francesco Ricci)

Recommender System Textbook